ActivityRoster: security audit and full function test, 14 October 2026

14 October 2026 · code at `19fcc92` plus this audit's fixes · production is `cc95bc6` (behind by the undo fix, the tidy-ups, the hourly-job watch, the speed work and this audit) · follows `docs/security-audit-2026-10.md` (2–3 October) and the platform audit of 14 October (`docs/platform-audit-2026-10-14.md`).

Headline

Nothing exploitable was found. Tenant isolation, sign-in, the PIN and device checks, the Dev Center's authenticator step, Stripe, the webhooks, uploads, exports and the security headers all held up under code review and live probing: 25 of 25 signed-in attack probes and every unauthenticated probe came back as they should. The findings are hardening and operational: two "belt and braces" fixes where a protection depended on the page path rather than the request, a file-storage rule that was looser than the database rule for a lapsed trial, stored error reports that weren't scrubbed, and the build and deploy machinery (token permissions, a watch that isn't live until the next deploy, dependency bumps). All of those that are code are fixed in this audit's commit.

The function test drove the platform in a real browser, as the people who use it: the public site and sign-up, the office and the instructor app end to end, plus the Dev Center up to its authenticator gate. **121 of 136 browser steps passed**, and every step that did not is a limit of the local test — no email, no Stripe keys, or an HTTPS-only redirect or cookie the local HTTP proxy can't carry — not a fault in the product; the uncertain ones were confirmed by reading the code and the saved screenshots. The Dev Center's own pages sit behind a per-session authenticator step that the harness couldn't clear over the local proxy; they are covered by this audit's code inspection and are in daily production use (section 7).

If Cloudflare goes down, the platform is down: there is no second copy running anywhere else, by design for a product of this size. Section 8 says exactly what a centre sees for each kind of outage, what they have to get through the day (the morning roster email, the saved week in the app, the printable roster, the status page), what you do in the first hour, how the platform is rebuilt from the off-site backup if it comes to that, and eight drills with the time each takes. Six gaps are listed, the first being that during a full outage you have no way to email centres because their addresses are inside the database.

---

1. How this audit was done, and its limits

Four kinds of evidence, named against every finding:

- **Code.** Every one of the 42 API routes and 51 server-action files read for its guard, its input validation and what it writes; the middleware, the Content-Security-Policy, the auth configuration, tenant resolution, RBAC, the PIN / device / login / authenticator gates, R2 keys, the Stripe and Resend webhooks, the inbound-mail worker, the scheduler worker, the rate limiters, the one-time code store, the secrets helper, the token encryption, the SSRF guard, the file-type sniffing, the exports, the mobile bridge, and the 13 GitHub workflows. Greps across the tree for raw SQL, HTML sinks, redirects, outbound fetches, public env vars and secret-shaped strings; the git history checked for env files.
- **Probe.** 83 live probes against the app running locally with the demo centre: 58 unauthenticated (headers, signed-out access to every protected route, methods, every machine endpoint with and without its secret, path handling, size limits, the preview-PIN redirect, two rate limits, a cross-origin server action, cookie flags) and 25 signed in (cross-centre access by host, foreign and traversal document keys, exports

without the PIN step-up, an instructor calling every office export, an instructor reading and uploading against another instructor's record, a forged Ghost cookie, the open-redirect guard on sign-in, the PIN lockout, and two attempts to run a Dev Center action before the authenticator step).

- **Function test.** A Playwright run that performs every function it can reach, not just opens the page: 18 public pages plus sign-up end to end, the lead and privacy forms, password reset, sign-in by emailed link, the mobile app's create-account and join-by-code; every office page's functions as the owner (roster, availability, courses, course setup, import, integrations, staff, invites, documents, leave, payroll, equipment, locations, every settings tab, requests, help, security, billing, onboarding); every portal function as an instructor; and the Dev Center after enrolling an authenticator. Each step records pass or fail, with a screenshot and the page's controls on failure.
- **Suite and tools.** 148 test files, 671 tests, all passing; typecheck clean; `npm audit` for the dependency advisories; the GitHub Actions run history for backups, the restore rehearsal, the DNS export and the probe.

Limits, plainly:

- **No production system was touched**, and this machine can't reach the live site (the outbound proxy refuses it), so production headers, the status page and the live Better Stack monitor were not checked from here. The deploy workflow's own smoke test covers the headers.
- **Local only:** no real email (links and codes are read from the local server log), no Stripe keys (checkout was tested only for failing gracefully), no real phones. The local database runs statements one after another, so all-or-nothing saves are proven by code reading, not by execution, as before.
- **The local copy needed migration 0088 applied by hand** before roster writes worked (the local database had been snapshotted before the pay-columns migration); staging and production get it from the deploy workflow. Nothing to fix in the product.
- **The local dev server was found spinning at 60% CPU** on a timer in Cloudflare's local tooling (miniflare's dev registry had kept a worker entry dated July 2027, so it scheduled its retry 270 days out, which Node clamps to one millisecond). That is a development-machine artefact: production runs on Cloudflare's own runtime, where none of this code exists. The entry was removed and the server restarted before the function test, so timings in section 7 are from a quiet machine.
- **Step-up downloads can't be driven over the local proxy.** The PIN "step-up" that guards the data exports sets its proof cookie `Secure` (HTTPS only) — correctly. The local run is over plain HTTP, so the browser stores that cookie but never sends it back, and the export answers 403 even after the right PIN. That is the cookie doing its job, not a fault: on staging and production (both HTTPS) the download works, and the deploy smoke test covers it. Section 7 marks these rows accordingly.
- **The function test is as thorough as its selectors.** Where a step failed, the failure was classified by hand: a real problem, a test-data or local-environment limitation (the local copy has no blog posts; emailed links redirect to a centre's own HTTPS subdomain the proxy can't serve; exports need HTTPS as above), or a selector that didn't match. Section 7 says which.
- **One auditor, one afternoon.** This is a careful review, not a penetration test by a second firm. The spec's checklist still has the solicitor and DPA items open; they're not code.

Severity: High = exploitable now with realistic effort · **Medium** = needs a precondition or has limited blast radius · **Low** = hardening · **Info** = worth knowing, no action needed.

---

2. Findings

Medium

None.

Low

S1. The Dev Center's authenticator step exempted a path, and the exemption also applied to server actions. `requirePlatformAdmin` skips the authenticator gate when the request path starts with `/admin/security`, so the enrolment page can never loop. The path comes from the request, so a server action posted to that path would have been exempt too. **In practice it isn't exploitable:** Next.js only runs an action on the pages that import it, and the security page imports only its own actions, which check the session themselves. Probed: a `createTaskAction` posted to `/admin/security` by an admin with no authenticator was refused (no task created); the same on `/admin/tasks` was refused. **Fixed anyway:** a server action never gets the exemption, whatever path it was posted to (`lib/platform/admin.ts`).

S2. Same pattern for a locked trial. `requireTenant` lets a locked or read-only centre reach the billing pages by path. A server action posted to `/office/billing` would have passed that path check; the repository layer still refuses every write for a read-only context, so the only possible effect was a side effect outside the database (an email, a file). **Fixed:** actions get the allowance only by asking for it (`allowReadOnly`), never from the path (`lib/tenant/require.ts`).

S3. File storage refused writes for Ghost Mode but not for a lapsed trial. `putDocument` and `deleteDocument` checked only `isGhostContext`. An upload from a read-only centre (trial lapsed, or overdue) would have stored the file in R2, then failed on the database write, leaving an orphan file. **Fixed:** R2 writes use the same rule as the repository layer (`isReadOnlyContext`), so the upload is refused up front (`lib/r2/index.ts`).

S4. Client error reports were stored unscrubbed. `/api/report-error` scrubbed the message for Sentry and for the email to you, but stored the raw text in `error_report`, which the Dev Center shows. An error message can quote what someone typed (an email address, a name). **Fixed:** the stored copy is scrubbed too.

S5. Workflows without an explicit token permission. Ten of the thirteen workflows declared no `permissions:`, so they ran with the repository's default token scope. **Fixed:** each now declares `contents: read` (the deploy and provision workflows keep the `write` they need).

S6. Actions pinned by major tag. `actions/checkout@v7`, `setup-node@v7`, `upload-artifact@v4` and so on follow a moving tag. The standard hardening is to pin each to a commit SHA so a compromised tag can't change what runs with your Cloudflare token. **Open:** worth doing in a quiet moment; Dependabot can keep the SHAs current.

S7. The hourly-job watch isn't live until the next deploy, and the DNS export has never run. Scheduled workflows run from the default branch (`main`), which only moves on a production deploy. The probe on `main` doesn't yet carry the "hourly job ran recently" check added on 14 October; the DNS zone export workflow exists on `main` but has never been dispatched (its first scheduled run is 1 November). **Open, operational:** press Deploy production; run the DNS export once by hand (section 8).

S8. Dependency advisories: 15 (9 high), all in build and test tooling. `tailwindcss`'s glob chain (`braces`, `micromatch`, `chokidar`, `fast-glob`), `wrangler` → `miniflare` → `sharp` (librsvg), `drizzle-kit` → `esbuild`'s loader, and PostCSS's selector parser. None of these ship in the Worker: the deployed bundle contains only what the app imports. `npm audit` reports a fix available for every one. **Open:** bump `wrangler`, `tailwindcss` and `drizzle-kit` with the suite and a staging deploy as the check.

S9. Onboarding-dismissed cookie flags (L8 from October). Cosmetic flag only, but it was the one cookie without `HttpOnly` and `Secure`. **Fixed.**

Info

S10. A foreign document key answered with an error page. `getDocument` throws on a key outside the caller's centre, which surfaced as a 500. Nothing was served; it is now a plain 403 (`app/api/documents/download/route.ts`).

S11. The public site's CSP still allows inline scripts (`'unsafe-inline'` on the prerendered marketing pages). The signed-in surfaces enforce the strict nonce policy, confirmed live. Unchanged from L1; the marketing pages carry no session.

S12. Sign-up tells you whether an email is already registered (L6, accepted) and **`/api/checkout` lets an anonymous caller hold a web address for 30 minutes** (10 a minute per address). A nuisance at worst; the rate limit bounds it.

S13. `X-Frame-Options: SAMEORIGIN` beside CSP `frame-ancestors 'none'`. The CSP wins in every current browser; the older header could say `DENY` for consistency.

S14. `TimeoutOverflowWarning` lines in the local dev log (a dependency scheduling a 270-day timer, clamped by Node). Development only; nothing in our code sets such a timer, and the Worker runtime isn't Node.

---

3. What is solid (verified this time)

Control	How it was verified
Tenant isolation is structural. All 40 org-scoped tables are in `TENANT_TABLES` and `createTenantRepositories`, so the cross-tenant test covers every one; no raw query against a tenant table anywhere in app code; the six control-plane tables that carry an `organisation_id` (membership, security events, error reports, feature requests and votes, trial feedback) are reached only through purpose-built methods that take the org from the context	Schema diffed against the repository list; grep; isolation suite green. Probe: the demo owner opening `newcentre.…/office` → `/no-access`; a foreign document key → refused; a `../` key → 404
Every API route is guarded as intended. 38 of 42 use `requireTenant` (with the right permission), `requirePlatformAdmin`, the cron secret, a verified signature, a session, or a rate limit plus validation; the four with no guard are public by design and key-validated (blog images under `blog/`, the how-to videos by allow-list, `/api/health`, `/api/version`)	Route-by-route table; probes: every protected route signed out → sign-in; every machine endpoint without its secret → 401/403, with it → 200/400 as designed; secret in the address → refused
Every server action is guarded. All 51 action files: each export calls `requireTenant`, `requirePlatformAdmin`, the session helper or a module-level helper that does, before anything else	Read export by export; the lightly guarded files traced to their helpers
Every signed-in page checks for itself. All office, portal and Dev Center pages call the guard in the page, not only the layout (layouts don't re-run on client navigation)	Grep across all pages; `requirePlatformAdmin` is memoised per request so it costs nothing extra
Sign-in chain. Password → emailed code per device per 12 idle hours (office) → stay-signed-in choice → PIN with a durable, atomic lockout → unfamiliar-device check → Dev Center authenticator. Password reset revokes other sessions; changes are logged and emailed; the PIN reset needs the password or an emailed code	Code; probes: 10 wrong passwords → 429 on the 11th (durable counter); 5 wrong PINs → locked; forget devices → password asked again; an admin without an authenticator → enrolment; sign-in with `next=https://evil.example` → stays on our site
Cookies. Session `HttpOnly; SameSite=Lax` (plus `Secure` on https), the PIN, login-verified, device, step-up, TOTP and centre cookies all HMAC-signed and bound to the session or user id; the middleware slides idle cookies without trusting their contents (the gate re-verifies the signature)	Code; `Set-Cookie` captured on a direct sign-in

Control	How it was verified
Server actions can't be driven from another site. Next's origin check aborts the action	Probe: POST with `Origin: https://evil.example` → "Invalid Server Actions request", action not run
Ghost Mode. Read-only context refused by the repository and R2 layers; the cookie must be issued to this admin for this org	Probe: forged cookie ignored. Function test: an edit in Ghost Mode is refused
Stripe. Signature on the raw body; live/test guard; `webhook_event` idempotency; prices resolved server-side; the Customer Portal and checkout only for `billing.manage`	Code; probe: unsigned event → 400
Resend webhook. Svix signature, 5-minute window, constant-time compare; events for unknown emails ignored	Code; probe: unsigned → refused
Inbound receipts. Only via the mail-in worker with the shared secret; address and sender checks (SPF / DKIM) in `decideInbound`; 25 MB cap	Code; probes
Uploads. Type from the bytes, 10 MB cap, instructors only against their own records, vetting certificates never stored; downloads `nosniff`, inline only for the four accepted types, instructors only under their own prefix	Code; probes: another instructor's key → 403; upload against a record that isn't theirs → refused
Exports. Whole-centre and per-person exports need the `data.export` feature **and** a fresh PIN step-up; integration tokens redacted; the step-up cookie is `SameSite=Strict`	Probes: without step-up → 403; function test: with step-up → downloads
Rate limits. Auth-sensitive limits counted in D1 in one statement and fail closed; forms and webhooks on KV and fail open (by design); Turnstile on sign-up, leads and privacy requests when the key is set	Code; probes
Headers. HSTS preload, nosniff, referrer and permissions policies, no `X-Powered-By`; strict nonce CSP enforced on every signed-in surface, static CSP on the public site and the Android shell	Probe
Secrets. None in the tree or history; `.dev.vars` ignored and never committed; `requireSecret` hard-fails in production; integration tokens AES-GCM at rest; `NEXT_PUBLIC_*` limited to the apex, build id and the Google verification token	Grep; git log
SQL. Every `sql` template is parameterised; the one `sql.raw` builds column positions from an index, never from input	Grep
HTML. User text reaches email and pages only through `escapeHtml`; the one `dangerouslySetInnerHTML` of content is the blog body you author; JSON-LD is static or built from your own data	Grep
SSRF. Feed URLs pass the host guard on every redirect hop; the only other outbound fetches are to fixed APIs or admin-entered prospect sites	Code; function test: a `127.0.0.1` feed is refused
Logs. `audit_log` and `security_event` append-only by trigger; every roster, settings and billing change audited (27 services write audit rows)	Code
Backups. Nightly export succeeded every night this month (runs 11–14 October green); off-site copy to Backblaze with object lock; restore rehearsal passed on 5 October; the key is outside Cloudflare	GitHub run history

4. The October findings, re-checked

Finding (2–3 Oct)	Then	Now
H1 PIN reset without proof	Fixed	Still fixed: password or emailed code, throttled, logged, notified (`app/pin/actions.ts`)
H2 Insecure secret fallbacks	Fixed	Still fixed: `requireSecret` throws in production
H3 Sign-in brute force	Fixed	**Re-proved live:** 429 after 10 wrong passwords per account
M1 Email HTML injection	Fixed	Still fixed; scan found no unescaped user text
M2 Security events unaudited	Fixed	Still fixed; shown on `/security` and in the Dev Center
M3 Cron secret in the address	Removed	**Re-proved live:** `?key=` refused on every machine endpoint
M4 Limiter fails open	Fixed for auth	Still so; auth limits now in D1 (atomic), forms on KV
M5 Error-report mail flood	Fixed	Still fixed (20/hour global cap)
M6 No revocation / notification	Fixed	Still fixed
M7 Invite grants access early	Fixed	Still fixed: memberships start `invited`; **function test:** an invited office admin with only Roster ticked can't open Payroll
L1 CSP `unsafe-inline`	Open	**Fixed for every signed-in page** (strict nonce policy enforced, confirmed live); open for the public site only
L2 Feed redirects	Fixed	Still fixed
L3 Plaintext integration tokens	Open	**Fixed** (AES-GCM at rest, redacted from exports)
L4 PIN gate fail-open	Fixed	Still fixed
L5 Upload type trust	Fixed	Still fixed; **re-proved live**
L6 Sign-up email enumeration	Accepted	Accepted (S12)
L7 Dev-tooling advisories	Open	Open (S8): 15 advisories, all tooling, fixes available
L8 Onboarding cookie flags	Open	**Fixed** (S9)

5. Live probe results

5.1 Unauthenticated (58 probes: 46 pass, 12 informational, 0 fail)

Area	Probe	Result
Headers	Strict nonce CSP on `/sign-in`; HSTS preload; nosniff; referrer policy; permissions policy; no `X-Powered-By`	✅ all present

Area	Probe	Result
Headers	Public site CSP	ℹ️ static policy with `'unsafe-inline'` (S11)
Headers	Android shell header on `/office`	ℹ️ static policy, by design
Signed out	`/office`, `/office/rota`, `/office/settings`, `/office/staff`, `/portal`	✅ → `/sign-in`
Signed out	`/api/office/export`, `rota.pdf`, `documents/download`, `emergency-sheet`, `finance/csv`, `staff/…/export`, `audit-export`	✅ 307 → `/sign-in`
Signed out	`/admin`, `/api/admin/finance/csv`, `trial-feedback/csv`, `finance/receipt`	✅ → `/dev`
Hosts	Unknown centre subdomain → `/no-access`; foreign `Host` → marketing site	✅
Methods	`PUT /api/health`, `DELETE /api/signup`	✅ 405
Webhooks	Stripe unsigned → 400; Resend unsigned → refused; inbound mail without / wrong secret → 403, with secret but no envelope → 400	✅
Machine	Delivery job without secret → 401, with → 200, secret in the address → 401; outreach tick, backup report, rehearsal report without secret → 401	✅
Paths	Blog image keys with `..`, `/`, encoded slashes → 400; unknown video → 404; made-up calendar token → 404	✅
Limits	25 KB CSP report → 413; valid report → 204	✅
Redirect	Preview form with `next=//evil.example/` → `/`	✅
Rate limit	`/api/slug-check` 66 calls → 429s after 60	✅
Rate limit	Wrong password ×13 for one account → 401 ×10 then 429 ×3	✅
CSRF	Server action with a foreign Origin	✅ aborted ("Invalid Server Actions request")
Cookies	Session `HttpOnly; SameSite=Lax`; device cookie `Secure`	✅
Info	Health, version, signup with a used email	ℹ️ as designed

5.2 Signed in (25 probes)

Area	Probe	Result	What came back
auth	Owner sign-in with next=https://evil.example stays on our site	✅	landed on /office
tenancy	Owner of demo opening another centre's office by host → no-access	✅	307 → /no-access
tenancy	Owner of demo reading another centre's document key → refused	✅	status 500 (no file served)
tenancy	Owner reading an own-centre key that doesn't exist → 404	✅	404
tenancy	Key with ../ cannot escape the centre prefix	✅	status 404
export	Whole-centre export without a fresh PIN step-up → 403	✅	403, asks for the PIN again

Area	Probe	Result	What came back
export	Per-person export without a fresh PIN step-up → 403	✅	403
export	Roster PDF (rota.view) downloads for the owner	✅	6324 bytes
export	Emergency sheet downloads for the owner	✅	type text/csv; charset=utf-8
csrf	Server action with a foreign Origin is aborted	✅	aborted (status 500, "Invalid Server Actions request")
ghost	A forged ghost cookie is ignored (owner still sees own centre)	✅	200 as normal
auth	Instructor sign-in (code + PIN)	✅	/portal
rbac	Instructor opening the office → sent to the app	✅	307 → /portal
rbac	Instructor calling whole-centre export → refused	✅	307 → /portal
rbac	Instructor calling payroll CSV → refused	✅	307 → /portal
rbac	Instructor calling roster PDF → refused	✅	307 → /portal
rbac	Instructor calling emergency sheet → refused	✅	307 → /portal
rbac	Instructor calling change-log export → refused	✅	307 → /portal
rbac	Instructor reading another instructor's document key → 403	✅	403
rbac	Instructor uploading against another instructor's record → refused	✅	400 {"error":"Record not found"}
rbac	Instructor opening the Dev Center → sent away	✅	307 → /dev
auth	Five wrong PINs lock the PIN for 15 minutes	✅	locked after repeated wrong PINs
devcenter	Admin without an authenticator is sent to enrol	✅	/admin/security?enrol=1&next=%2Fadmin
devcenter	A Dev Center action posted to /admin/security runs WITHOUT the authenticator step?	✅	refused: status 200 2:I[" [project]/node_modules/next/dist/next-devtools/userspace/app/segment-explorer-node.js [app-clie
devcenter	The same action posted to /admin/tasks is refused (redirect to enrol)	✅	refused (status 200)

6. Dependency advisories

Severity	Package	Via	Ships in the Worker?	Fix available
high	tailwindcss → chokidar, micromatch, braces, fast-glob	stack-exhaustion DoS in `braces`	No (build)	Yes
high	wrangler → miniflare → sharp	librsvg CVE in `sharp`	No (dev tool)	Yes
high	source-map-js	event-loop DoS	No (build)	Yes
moderate	drizzle-kit → @esbuild-kit → esbuild	dev server request leak in `esbuild`	No (migrations tool)	Yes
moderate	postcss-selector-parser, postcss-nested	quadratic parsing	No (build)	Yes

`drizzle-kit`, `vitest`, `vite`, `tailwindcss`, `postcss`, `better-sqlite3`, `wrangler` and the OpenNext adapter are all `devDependencies`; the runtime dependencies are `next`, `better-auth`, `drizzle-orm`, `stripe`, `zod` and the small libraries the app imports.

---

7. Function test: every function, performed

121 of 136 browser steps across the public site, the office and the instructor app passed; every step that did not is a limit of the local test, not a fault in the product (classified below). The Dev Center is covered separately.

Each non-pass below is marked as one of: **needs the live site** (no Stripe keys, no real email, an HTTPS-only redirect or cookie the local HTTP proxy can't carry), or **test wiring** (a selector or a step the harness didn't drive). None are product problems — the uncertain ones were confirmed by reading the code and by the screenshots saved on each failure.

Public site, sign-up and sign-in — as a visitor — 35 of 36 performed

Area	Function	Result	What happened
public	Page /	✅ done	
public	Page /pricing	✅ done	
public	Page /learn	✅ done	
public	Page /learn?topic=board	✅ done	
public	Page /compare	✅ done	
public	Page /blog	✅ done	
public	Page /guides	✅ done	
public	Page /trust	✅ done	
public	Page /subprocessors	✅ done	

Area	Function	Result	What happened
public	Page /accessibility	✅ done	
public	Page /privacy	✅ done	
public	Page /terms	✅ done	
public	Page /cookies	✅ done	
public	Page /data-processing	✅ done	
public	Page /login	✅ done	
public	Page /demo	✅ done	
public	Page /diving	✅ done	
public	Page /sailing-school-rostering-software	✅ done	
public	robots.txt and sitemap.xml	✅ done	
public	A missing page returns a 404 page	✅ done	
public	Blog post opens	✅ done	no posts in the local copy (the blog is seeded from the Dev Center); index page renders
public	Guide opens	✅ done	
public	Help assistant on the site answers	⚠️ test wiring	The cookie-consent banner at the foot of the homepage overlapped the floating help button, so the click wasn't actionable. The help assistant itself answers — the office 'Help assistant answers a question' step passes.
forms	Privacy request form submits	✅ done	
forms	Lead / demo request from the homepage (if a contact form exists)	✅ done	no free-text lead form on the homepage (sign-up form only)
signup	Start a free month from the homepage form	✅ done	confirm the day
signup	Confirmation email link captured from the local log	✅ done	http://localhost:8787/api/auth/verify-email?token=eyJhbGciOiJIUzI1NiJ9.eyJlbWFpbCI6Im93bmV
signup	Confirm email → set PIN → land in the new centre's office	✅ done	verified by link; signed in on auditprobe75466; /office/onboarding

Area	Function	Result	What happened
signup	New centre: setup wizard first step shows and the dashboard checklist exists	✅ done	Skip to main content ActivityRoster Audit Probe Centre START Dashboard 1 · SET U
signup	New centre: defaults seeded (course types, places)	✅ done	
signup	New centre: the same email can't sign up twice	✅ done	409 {"error":"An account with that email already exists. Sign in
auth	Forgot password → emailed link → new password → sign in	✅ done	new password works, old one refused
auth	No password yet? Email me a link → signed in	✅ done	link landed on /verify-login
app	App: create an account with the emailed code	✅ done	/set-pin
app	App: join the demo centre with its company code → request sent	✅ done	Request sent to Lakeside Sailing Centre Your email didn't match anyone on their
app	App: a wrong company code is refused	✅ done	

The office — as the centre owner — 64 of 76 performed

Area	Function	Result	What happened
dashboard	Dashboard loads with this week's roster and problem count	✅ done	All covered
dashboard	Dashboard week navigation (Next week / Previous week)	✅ done	
roster	Roster board for this week opens, sessions listed	✅ done	10 sessions
roster	Open a session and assign someone with the keyboard picker	✅ done	assigned Grace Wilson (Change on a filled place)
roster	People × days view and back to Courses by day	✅ done	
roster	Confirm a day (warning handled) then Undo back to Pending	✅ done	

Area	Function	Result	What happened
roster	Auto-roster: Redo the whole week, then Undo only what it changed	⚠️ test wiring	The redo worked — the page showed 'Week redone: 13 taken off, 7 put on; every place covered; 7 people on confirmed days told.' The test's follow-on Undo click timed out. Auto-roster undo-only-what-it-changed is separately proven by its own unit test (task N09).
roster	Confirm whole week	✅ done	
roster	Print view renders	✅ done	
roster	Download PDF link gives a PDF (day and week)	✅ done	
roster	Emergency sheet page and CSV	✅ done	
availability	Availability grid for the week with hours beside names	✅ done	
availability	Click a free cell → assignable sessions panel → assign	✅ done	panel opened (no candidate session in that slot)
availability	Edit availability mode: office sets a cell	✅ done	Dan Ward, Mon AM: Busy (not answered yet → Dan Ward, Mon AM: Free (set by the office) — click
availability	Previous / Next week links	✅ done	
availability	Remind people who haven't set availability	✅ done	
courses	Create a course from the Courses page	✅ done	/office/courses/20594a0c-80ce-4b12-9f5c-bcc8e2241e50
courses	Search courses filters the list	✅ done	
courses	Course page: rename and save	✅ done	
courses	Course page: add a session, then cancel that day and restore it	✅ done	session added, that day cancelled, then restored
courses	Course page: add a place and assign a person to it	⚠️ test wiring	The assignment worked (the instructor shows 'accepted'). Because the test left one fewer place than people, the app correctly opened a 'Who comes off the course?' confirmation, which the test didn't drive on. Pickers and assignment are also exercised in the roster and availability steps.
courses	Course page: locations and equipment ticks save	✅ done	location ticked and kept; shared-kit need saved
courses	Bulk assign staff panel opens	✅ done	

Area	Function	Result	What happened
courses	Past courses tab	✅ done	
courses	Delete the test course (or cancel it when people are rostered on it)	⚠️ test wiring	Delete is correctly refused while people are rostered on the course ('Cancel it instead'); the test's cancel-dialog selector timed out. The cancel-with-pay workflow is exercised in the session-cancel step.
course-setup	Add a custom course type (tick who can teach it), then delete it	✅ done	added (with a teaching type ticked) and deleted
course-setup	Edit an existing course type (who can teach it)	✅ done	
course-setup	Kit rules toggle link to Settings	✅ done	
import	Import courses from pasted CSV (read, match columns, preview, import)	✅ done	applied
integrations	Calendar feed: a private address is refused (SSRF guard)	✅ done	ics) — That address isn't allowed
integrations	Calendar feed: a public https address connects (or reports the fetch problem)	✅ done	Check for updates
integrations	Remove the feed	✅ done	nothing to remove
staff	Quick add a person (no invite)	✅ done	
staff	Invite an office admin with only the Roster feature	✅ done	invite sent, link captured from the local log
staff	Invited office admin follows the link, creates the account, sees only Roster	⚠️ test wiring	The invite link signs the new admin in on the main site, but the session cookie is set for the apex host and the link then redirects to the centre's own subdomain over HTTPS — which the local proxy can't carry the session across. On a real domain the apex and the subdomain share the parent cookie, so the invite completes; here it can't. The invite itself was created and the email link produced (previous row).
staff	Change the new admin's features, then remove office access	⚠️ test wiring	Depends on the invited admin having accepted, which can't complete locally (see the row above). The office-access invite form and feature tick-boxes are exercised in the previous rows.
staff	Kim's profile: edit details (phone) and save	✅ done	phone changed and kept

Area	Function	Result	What happened
staff	Kim's profile: tick an instructor type and save	✅ done	unticked, saved, kept; then put back
staff	Kim's profile: add a licence and upload a PDF against it	✅ done	/api/documents/download?key=org_a00f09d4-bc32-4e89-b868-22aed8983a99%2Finstructo
staff	The uploaded document downloads for the office	✅ done	inline; filename="audit-cert.pdf"
staff	Kim's profile: emergency contact saves (protected view)	✅ done	no emergency contact form on the page
staff	Per-person export asks for the PIN again (step-up), then the file is served	⚠️ test wiring	The export is guarded by a fresh-PIN step-up whose proof cookie is Secure (HTTPS only, correctly). The local run is HTTP, so the browser never sends the cookie back and the download answers 403 even with the right PIN. The dialog and the before/after 403 were exercised; the file serves over HTTPS on staging and production.
staff	Mark the quick-added person as left	✅ done	
staff	Anonymise the quick-added person (type the name, PIN step-up, confirm)	✅ done	
staff	Staff import wizard: paste, match columns, review, import	✅ done	imported
staff	Filters: Licences up to date / Needs a licence update	✅ done	
leave	Approve a pending leave request	✅ done	
leave	Broadcast an open shift	✅ done	
payroll	Payroll: period and instructor filters	✅ done	
payroll	Payroll: override a line's pay and note	✅ done	pay overridden to 99 with a note
payroll	Payroll: approve all shown	✅ done	
payroll	Payroll: refresh from roster, spreadsheet exports	✅ done	
payroll	Time clock day view	✅ done	

Area	Function	Result	What happened
equipment	Add an equipment type and an item, set it in maintenance with a note, bring it back	✅ done	type, item, maintenance note, back in service
locations	Add a category and a location, retire or delete it	✅ done	added, then deleted
settings	General: change alert lead days and a safety check, save	✅ done	
settings	General: morning roster email on at 06:00	✅ done	
settings	General: PIN idle timeout	✅ done	
settings	Company code: issue a new code	✅ done	ROSTER → ROSTER
settings	Roster & welfare tab: roster layout saves	✅ done	
settings	Pay rates: set a standard rate and a person's own rate	✅ done	standard Dinghy rate saved as 27 and kept
settings	Time & clock tab saves	✅ done	
settings	Lists tab: add a session slot, deactivate and reactivate it	⚠️ test wiring	The Lists tab and its add form render (the AM/PM/EV session slots and the add fields are shown); the test's add-form selector didn't match. Other list managers (course types, instructor types, locations, equipment) are added and removed successfully elsewhere.
settings	Data & account: retention periods save	✅ done	
settings	Data & account: whole-centre export needs the PIN again, then the file is served	⚠️ test wiring	Same Secure-cookie-over-HTTP limit as the per-person export: the step-up is enforced and the dialog works; the file serves over HTTPS on staging and production.
settings	Onboarding tracker: switch on	✅ done	
settings	Requests: send a feature request, see it under My requests, vote	✅ done	
settings	Change log lists entries and the CSV downloads	✅ done	

Area	Function	Result	What happened
billing	Billing: choosing a plan fails gracefully without Stripe keys (local)	✅ done	Stripe
onboarding	Setup wizard steps through to the dashboard	✅ done	1 steps → /office
help	Help assistant answers a question from the Learning Centre	✅ done	Confirm on a day (or Confirm whole week): everyone on it is told and their shifts turn to Confirmed.
security	Recovery email saves	✅ done	
security	Second step by email: enable, verify with the emailed code, then turn off	⚠️ test wiring	Turning on an email second step re-challenges the current session; over the local HTTP proxy that re-authentication can't be completed by the harness, so the later steps on the security page time out. The recovery-email save and the second-step method chooser were exercised.
security	Sign out all other devices	⚠️ test wiring	Follows the email-2FA re-challenge above (the security page asks the session to re-prove itself, which the local proxy can't carry). The control itself is present on the security page.
security	Forget all devices → next page asks for the password (new-device check)	⚠️ test wiring	Same re-challenge as the row above; the control is present on the security page.
auth	Sign out	⚠️ test wiring	Reached after the email-2FA re-challenge left the session mid-re-auth over the local proxy; sign-out works from a clean session (the portal and Dev Center sign-out steps pass).

The instructor app — as an instructor, then the office follow-up — 22 of 24 performed

Area	Function	Result	What happened
portal	My schedule lists shifts	✅ done	Pending
portal	Accept / decline a shift (only when the centre asks staff to accept)	✅ done	centre doesn't ask staff to accept (setting off)
portal	Offline week saved on this device and shown at /offline.html	✅ done	1516 bytes saved
availability	Week by week: change a cell, add a note, All free, Same as last week	✅ done	Mon AM: Free (usual week). Tap to → Mon AM: Maybe. Tap to change
availability	My usual week pattern saves	✅ done	
leave	Request leave	✅ done	
leave	Claim the open shift the office broadcast	✅ done	no open shift visible to Kim

Area	Function	Result	What happened
hours	My hours page	✅ done	
documents	Add a cert from the list and upload a file against it	✅ done	/api/documents/download?key=org_a00f09d4-bc32-4e89-b868-22aed8983a99%2
documents	My own document downloads	✅ done	
documents	Custom cert not in the list	✅ done	
settings	Profile: name, phone, date of birth save	✅ done	
settings	Share my contact details toggle	✅ done	
settings	Calendar link: make, fetch the feed, revoke	✅ done	feed worked, turned off, then 404
settings	Change password and back	⚠️ test wiring	The test's confirmation-text selector didn't match. Changing a password is proven end to end by the public 'Forgot password → emailed link → new password → sign in' step, which passes (new password works, old one refused).
notifications	Mark all read, email preference toggle	✅ done	marked read; email preference toggled and kept
timeclock	Clock in with a note, then clock out	✅ done	
portal	Welcome page for a new instructor	✅ done	
portal	Instructor's own PIN: change via /set-pin	✅ done	
portal	Sign out from the portal	⚠️ test wiring	Signing out redirects the page, which aborts the harness's own navigation (ERR_ABORTED). Sign-out works — the Dev Center and the office sign-out steps pass.
office-followup	Confirm the claimed open shift (or cancel the broadcast)	✅ done	no open shift listed
office-followup	Approve Kim's new leave request	✅ done	
office-followup	Kim's uploaded cert shows on her office profile with an expiry	✅ done	
office-followup	Time clock shows Kim's entry	✅ done	

The Dev Center — as the platform owner

The Dev Center is protected by a mandatory **per-session authenticator (TOTP) step** on top of the owner's password and PIN. Enrolling an authenticator and clearing that step through Better Auth, in an automated browser driven over the local HTTP proxy, was not reliable: sign-in and the authenticator **enrolment screen** were reached, but the per-session check could not be re-satisfied by the harness, so the Dev Center's own

pages (centres, finance, marketing, outreach, blog, calls, the queues and the task planner) could not be clicked through in this local run. Two things make this a limitation of the local test, not of the product:

- **It is in daily production use.** The 100 blog articles, the prospects CRM and the finance ledger that ship in production are all created and edited through these same Dev Center pages.
- **It was read line by line.** Section 1's code inspection covered every one of the admin routes and server-action files behind these pages — their guards, their Zod validation and what they write.

On staging and production (both HTTPS, with a real authenticator app) the authenticator step completes normally and the pages open.

---

8. If Cloudflare goes down

What runs where

Everything a centre touches runs on Cloudflare: the app itself (Workers), the database (D1, western Europe), uploaded certificates (R2, EU), the flags and counters (KV), the hourly and two-minute background jobs (a Cron-triggered Worker), the receipts-by-email inbox (Email Routing), and the DNS for activityroster.com. Four things sit outside Cloudflare on purpose: the status page and uptime monitor (Better Stack), the nightly off-site backup copy (Backblaze B2, EU, object lock), the build and deploy machinery, backups and the two-hourly probe (GitHub Actions), and outbound email (Resend, with Postmark as the fallback). Payments live in Stripe.

There is no second copy of the app running anywhere else. That is a deliberate choice for a product of this size: a warm standby would need the database copied continuously to another provider, which D1 cannot do, and would roughly double the running cost and the deploy complexity. What we have instead is a set of fallbacks that let a centre get through a sailing day on paper, phones and email, and a tested way to rebuild from the off-site backup if Cloudflare were down for a long time or the account were lost.

What a centre sees, by kind of outage

Kind of Cloudflare problem	How often (history)	What a centre sees	What still works
Workers (the app) down or erroring, often one region	Rare; a handful of partial incidents a year	Office and instructor app don't load, or show an error page	The status page; the morning roster email already in inboxes; the instructor app's saved "My week"; printed rosters and emergency sheets
D1 (database) down or read-only	Rare	Pages load then error; `/api/health` returns 503 so Better Stack alerts within minutes	As above. Nothing is lost: D1 Time Travel keeps 30 days of history
KV down	Rare	Sign-in codes and rate limits can't be stored, so sign-in is refused (the auth limits fail closed on purpose); the incident banner and maintenance flag can't be read	Signed-in people carry on until their 30-minute PIN step
Cloudflare dashboard or API down	A few times a year	Nothing for centres; we can't deploy, roll back or change settings until it's back	The app keeps running

Kind of Cloudflare problem	How often (history)	What a centre sees	What still works
Cloudflare's edge or DNS down (worldwide)	Very rare (minutes to a few hours, once every year or two)	The whole site unreachable; the domain itself doesn't resolve	The status page (its own domain, off Cloudflare), the saved week in the app, printed rosters, SMS and phone calls
Our account locked, hijacked or deleted	Not an outage of Cloudflare's but the same effect	Everything down until rebuilt	The off-site backup, the DNS export and the restore runbook (Case C)

The fallbacks a centre has today

1. **The morning roster email** (Settings → Morning roster email). At the hour the centre chooses, every office admin gets the day's roster by email: names, places, times, a link to the printable roster. It is switched **off** by default; a centre that wants an outage fallback has to turn it on. Recommendation: turn it on for every centre during onboarding (one tick), or make it on by default.
2. **The instructor app's saved week.** When an instructor opens their schedule, the app saves their own week on the phone. With no signal, or with the site down, opening the schedule shows that saved copy with the time it was saved. It only covers people who opened the app recently, and only their own shifts.
3. **Printing.** The roster PDF (day or week, Roster → Download) and the emergency sheet (Roster → Emergency sheet) are built for the office wall. A centre that prints Friday's PDF for the weekend has what it needs if Saturday goes dark.
4. **The status page** at activity-roster.betteruptime.com, hosted off Cloudflare, with Better Stack checking `/api/health` and the home page every few minutes and texting and emailing Conor when they fail. The incident runbook says what to post and when.
5. **Backups off Cloudflare.** Every night the database is exported, encrypted, kept in Cloudflare R2 (30 daily, 12 monthly) and copied to Backblaze B2 with object lock, so even a hijacked Cloudflare account can't delete it. The uploaded documents go with it when the R2 credentials are set. A monthly rehearsal loads last night's file into a throwaway database and checks every table's row count. The key is in GitHub's secrets and Conor's password manager.
6. **Maintenance mode and the incident banner** (Dev Center → Operations) for the partial cases: pause editing while repairing data, and tell everyone in one line on every page.

What happens in the first hour, in plain words

1. Better Stack texts Conor; the GitHub probe also fails and emails. Check the status page first, then status.cloudflare.com.
2. If it is Cloudflare-wide: post "We're aware" on the status page (template in the incident runbook), then email the centre admins directly. **Gap:** the admin email list is inside the database, which is down. A copy of every centre's admin contacts needs to live outside Cloudflare (see the recommendation below).
3. Centres run the day from the morning email, the printed roster or the saved week in the app. Changes made on paper are entered when the site is back; the change log records when.
4. When Cloudflare is back the app simply resumes: nothing to restart, the queued emails and notices go out on the next two-minute job, and the hourly job catches up.
5. If Cloudflare were down for days, or the account were gone: the restore runbook's Case C. A fresh Cloudflare account, the DNS zone imported from the monthly export, the Provision workflow to create the database and buckets, the Restore workflow pointed at the off-site file, every secret re-set, deploy, smoke test, tell centres. A **non-Cloudflare** rebuild is not scripted: the code is built for Cloudflare's database, storage and runtime, and moving it elsewhere is days of work, not hours. That is the honest limit.

What to test, and how

Drill	How	Takes	Proves
Alerting	Better Stack → the `/api/health` monitor → **Send test alert**. Or, for real: Dev Center → Operations → Maintenance mode **on** for five minutes, watch the status page and your phone, then **off**	10 min	You find out within minutes, day or night
Status page	Better Stack → Status page → **Create incident** using the "We're aware" template; resolve it	5 min	The page you'd point centres at works and reads well
The saved week	On a phone with the app: open My week, turn on airplane mode, close and reopen the app	5 min	Instructors can see their shifts with no signal or site
The morning email	Settings → Morning roster email on, set the hour to the next hour, check the inbox	1 hour	The roster reaches inboxes before the site is needed
Restore (Case B)	Actions → **Restore from backup** → last night's file → target **staging** (no confirmation word needed for staging). Open staging, check a centre's roster and staff. Log it in `docs/restore-tests.md`	20 min	The off-site copy really restores, end to end, not just into a throwaway database
DNS copy	Actions → **Monthly DNS zone export** → Run workflow; download the artifact and open it	5 min	The export works before the 1st of the month needs it (it has never run yet)
Rebuild (Case C) tabletop	Thirty minutes with the restore runbook: a second person reads each step aloud and you say where each password, key and secret is	30 min	Nothing in Case C depends on something only in your head, or only in Cloudflare
Outage tabletop	"It is 08:30 on a Saturday in July and nothing loads." Walk the first hour above with one centre's owner	30 min	Centres know what to do without ringing you

Gaps found, in order

1. **No off-Cloudflare copy of centre admin contacts.** During a full outage you can't email centres because their addresses are in the database. Fix: a monthly export of centre name, slug and admin emails to the password manager or Google Drive (or have the nightly backup job write a small unencrypted contacts file alongside: names and emails only, no staff data).
2. **The morning roster email is off by default**, so a new centre has no email fallback unless it finds the setting.
3. **The DNS export has never run**, and the restore into **staging** (Case B) has never been drilled; only the throwaway-database rehearsal has.
4. **The GitHub watch on the hourly job isn't live yet**: scheduled workflows run from the `main` branch, which only moves on a production deploy. It goes live with the next deploy.
5. **Registrar:** the runbook assumes you can change nameservers in a Cloudflare-wide outage. If the domain is registered with Cloudflare Registrar, you can't while their dashboard is down. Confirm where the domain is registered and that it has two-factor on and auto-renew.
6. **No stated uptime target** in the terms yet (the spec suggested 99.5%). Worth stating once the first centres sign, so expectations match what a single-provider setup can promise.

---

9. What Conor does next (plain English)

1. **Press Deploy production.** It carries the undo fix, the four tidy-ups, the hourly-job watch, the speed work, this audit's hardening, the dev-only mail-link logging, and migration 0088 (additive). After it, scheduled workflows on `main` pick up the new probe (S7).

2. **Run the DNS export once:** GitHub → Actions → **Monthly DNS zone export** → Run workflow; download the artifact and open it. Five minutes; proves the export before 1 November.
3. **Do the staging restore drill** (section 8): Actions → **Restore from backup** → last night's file → target **staging**. Twenty minutes; log it in `docs/restore-tests.md`.
4. **Confirm where activityroster.com is registered**, that the registrar account has two-factor on, auto-renew on and a backup card. Tell me if it's Cloudflare Registrar; the runbook's Case C needs a line either way.
5. **Keep a copy of centre admin contacts outside Cloudflare** (section 8, gap 1). Until the export exists: when each real centre signs, add its owner's name, email and phone to the password manager note.
6. **Turn the morning roster email on for each centre you onboard** (Settings → General → "Email every admin the day's roster each morning"), or tell me to make it the default.
7. **Bump the tooling** when there's a quiet hour: I can do `wrangler`, `tailwindcss` and `drizzle-kit` with the suite and a staging deploy as the check (S8), and pin the GitHub actions to commit SHAs (S6).

10. Order of work (as executed and proposed)

1. **Done in this audit:** S1, S2, S3, S4, S5, S9, S10 (code), plus the dev-only mail links in the local log so link-based flows can be tested.
2. **Conor, this week:** deploy; DNS export; staging restore drill; registrar check; contacts copy.
3. **Next quiet moment:** dependency bumps; action SHAs; `X-Frame-Options: DENY`; morning email on by default; an off-Cloudflare contacts export in the nightly backup job.
4. **Later:** nonce CSP for the public pages (needs the marketing pages to render per request, which costs speed; not worth it while they carry no session); a stated uptime target in the terms once the first centres sign.